Bottom-up Syntax Analisys
ELR (*k*) Method – I

Translated and adapted by L. Breveglieri

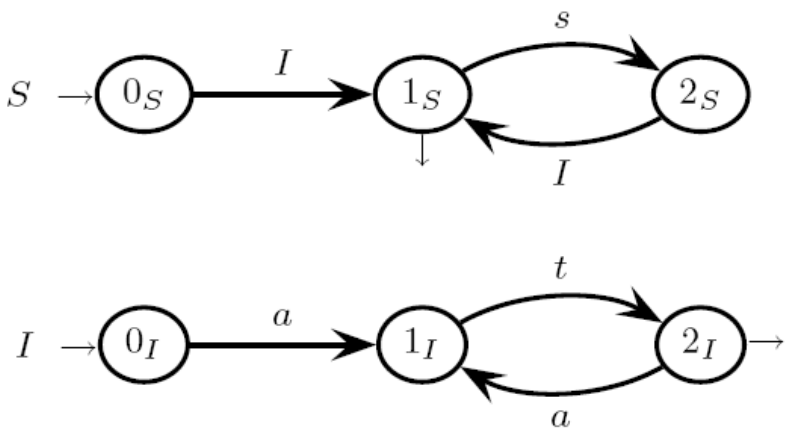
INFORMAL EXAMPLE – INTUITION ON THE BOTTOM-UP SYNTAX ANALYSIS

EBNF grammar

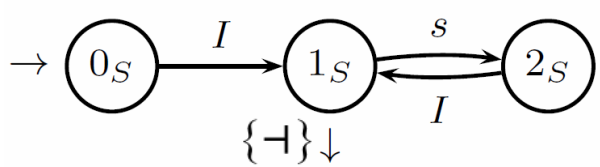
$S \rightarrow I \ (\ s \ I \ )^*$

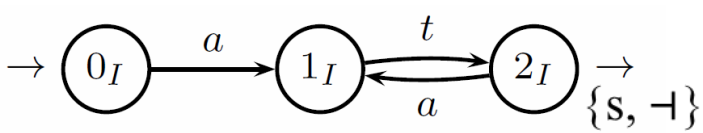
$I \rightarrow (\ a \ t \ )^+$

machine net



For the analysis add the *follow sets* of the non-terminals: *follow* (*S*) and *follow* (*I*)
the *follow set* is often called *look-ahead set* or *prospect set*





The follow sets allow us to decide when to

- go on with the analysis and run an automaton transition (*shift* move)
- or recognize a non-terminal and build a sub-tree (*reduce* move)

PARSING TRACE OF STRING $x = a\ t\ a\ t\ s\ a\ t \dashv$

Push a *macro-state* (m-state) $\{0_S, 0_I\}$, which contains the initial states of M_S and M_I

The analysis of axiom S starts from I: a kind of ε-move called *closure* or *completion*

stack bottom	string to be parsed (with end-marker) and stack contents								effect after	#
	1	2	3	4	5	6	7			
$\{0_S\ 0_I\}$	a	t	a	t	s	a	t	$\dashv$	initialization of the stack	1

current char *cc*

Since from state 0_I there is an *a*-arc and *cc* = *a*, make a *shift*

	a	t	a	t	s	a	t	$\dashv$		
$\{0_S\ 0_I\}$	a $\{1_I\}$								terminal shift on a: $0_I \xrightarrow{a} 1_I$	2

push *a* to remember the *a* read in the input

stack top

new current char *cc*

Since from state 1_I there is a *t*-arc and *cc* = *t*, make a *shift*

	a	t	a	t	s	a	t	$\dashv$		
$\{0_S\ 0_I\}$	a $\{1_I\}$	t $\{2_I\}$							terminal shift on t: $1_I \xrightarrow{t} 2_I$	3

from before

	a		t		a	t	s	a	t	$\dashv$			
$\{0_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$							terminal shift on t: $1_I \xrightarrow{t} 2_I$	3

Choice between *shift* and *reduce*: since it is *cc* = *a* $\notin$ *follow* (*I*), make a *shift*

	a		t		a		t	s	a	t	$\dashv$	(do not reduce)	
$\{0_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$	a	$\{1_I\}$						terminal shift on a: $2_I \xrightarrow{a} 1_I$	4

Since from state 1_I there is a *t*-arc and *cc* = *t*, make a *shift*

	a		t		a		t	s	a	t	$\dashv$		
$\{0_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$	a	$\{1_I\}$	t	$\{2_I\}$				terminal shift on t: $1_I \xrightarrow{t} 2_I$	5

from before

	a		t		a		t		s	a	t	$\dashv$		
$\{0_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$	a	$\{1_I\}$	t	$\{2_I\}$					terminal shift on t: $1_I \xrightarrow{t} 2_I$	5

Choice between *shift* and *reduce*: since it is *cc* = *s* $\in$ *follow* (*I*), make a *reduce*

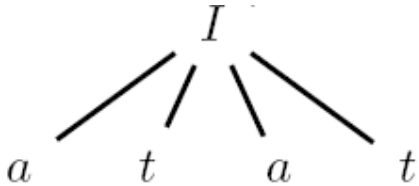
To find the reduction handle, explore the stack back to the initial state 0_I of M_I and pop all the explored portion (except 0_I) – easy strategy to be refined later

$$\overbrace{\{\,0_S\ 0_I\,\}\ a\ \{\,1_I\,\}\ t\ \{\,2_I\,\}\ a\ \{\,1_I\,\}\ t\ \{\,2_I\,\}}^{\textit{segment (handle) to be reduced by: } a\,t\,a\,t \rightsquigarrow I}$$

	I								s	a	t	$\dashv$		
$\{0_S\ 0_I\}$													since s follows I reduce $a\,t\,a\,t \rightsquigarrow I$	6

it does not overwrite the input tape (which is read-only) – it helps us remember the reduction operated

built a fragment of syntax tree



from before

	I	s	a	t	$\dashv$		
$\{0_S\ 0_I\}$						since s follows I reduce $a\,t\,a\,t \rightsquigarrow I$	6

Push the identified reduction *a t a t* → *I* : make a *non-terminal shift*

	I		s	a	t	$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$					nonterminal shift on I: $0_S \xrightarrow{I} 1_S$	7

Choice between *shift* and *reduce*: since it is *cc* = *s* ∉ *follow* (*S*), make a *shift*

	I		s	a	t	$\dashv$	(do not reduce)	
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s $\{2_S\ 0_I\}$				terminal shift on s: $1_S \xrightarrow{s} 2_S$	8

Then make two more *shifts* (here left uncommented)

	I		s		a		t	$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$	a	$\{1_I\}$			terminal shift on a: $0_I \xrightarrow{a} 1_I$	9

	I		s		a		t		$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$		terminal shift on t: $1_I \xrightarrow{t} 2_I$	10

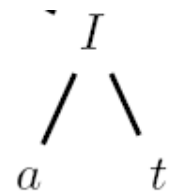
from before

	I		s		a		t	$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$	a	$\{1_I\}$	t	$\{2_I\}$		terminal shift on t: $1_I \xrightarrow{t} 2_I$	10

Choice between *shift* and *reduce*: since it is *cc* = ⊣ ∈ *follow* (*S*), make a *reduce*

	I		s		I	$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$			since $\dashv$ follows I reduce $a\,t \rightsquigarrow I$	11

built a fragment of syntax tree



Push the identified reduction *a t* → *I* : make a *non-terminal shift*

	I		s		I	$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$	I	$\{1_S\}$		nonterminal shift on I: $2_S \xrightarrow{I} 1_S$	12

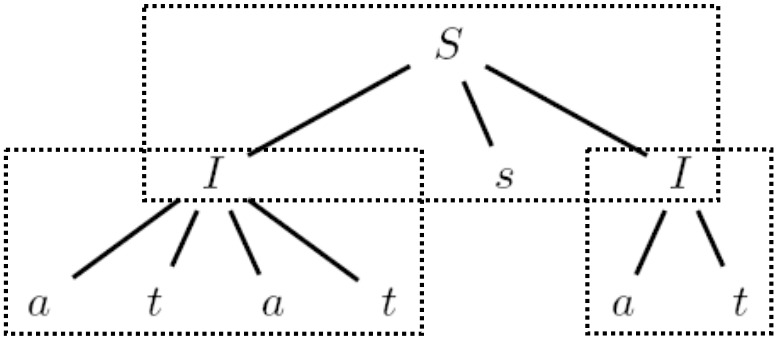
from before

	I		s		I		$\dashv$		
$\{0_S\ 0_I\}$	I	$\{1_S\}$	s	$\{2_S\ 0_I\}$	I	$\{1_S\}$		nonterminal shift on I: $2_S \xrightarrow{I} 1_S$	12

Choice between *shift* and *reduce*: since it is *cc* = ⊣ ∈ *follow* (*I*), make a *reduce*

	S	$\dashv$	
$\{0_S\ 0_I\}$		since $\dashv$ follows I reduce $I\ s\ I \rightsquigarrow S$	13

built the complete syntax tree



previous fragment

previous fragment

Analysis end and acceptance condition

	the input string is reduced to the axiom S	$\dashv$	
$\{0_S\ 0_I\}$	the stack contains only the axiom $\{0_S\ 0_I\}$	stop and accept	14

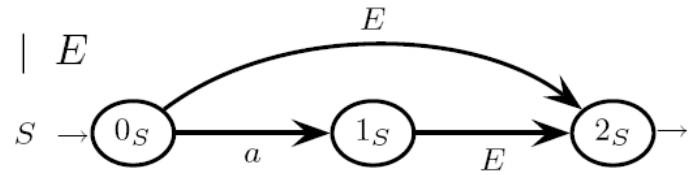
It is not easy to find the right reduction – it may be necessary to scan the string at a long distance

example with another grammar

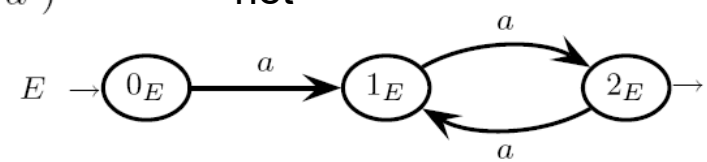
$S \rightarrow a\ E \mid E$

$E \rightarrow (\ a\ a\ )^+$

the strings of even length $(aa)^+ = L\ (0_E)$
and odd $a\ (aa)^+ = a\ L\ (1_S)$ are derived
in different ways



net



Derivation identifiable only when meeting the terminator

Analysis of string "*aaa*": in three steps the stack contains

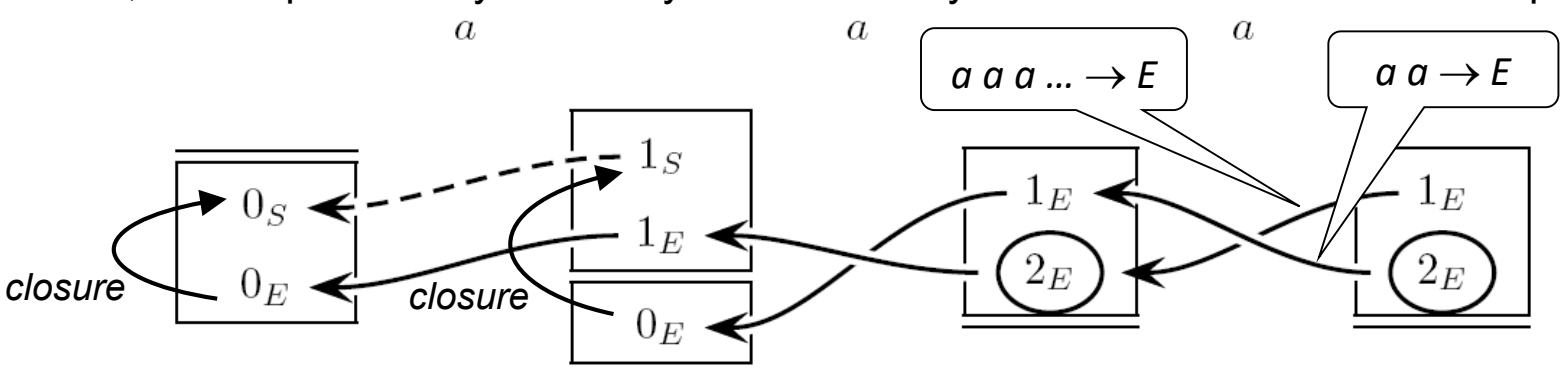
	$\{\ 0_S\ 0_E\ \}\ a\ \{\ 1_S\ 1_E\ 0_E\ \}\ a\ \{\ 2_E\ 1_E\ \}\ a\ \{\ 1_E\ 2_E\ \}$

The second m-state $\{1_S, 1_E, 0_E\}$ indicates three computations

$$0_S \xrightarrow{a} 1_S \qquad 0_S \xrightarrow{\varepsilon} 0_E \xrightarrow{a} 1_E \qquad 0_S \xrightarrow{a} 1_S \xrightarrow{\varepsilon} 0_E$$

From which 0_E do we reduce ? from $\{0_S,\ 0_E\}$, i.e., *aaa* … → *E*, or from $\{1_S, 1_E, 0_E\}$, i.e., *aa* → *E* ?

To decide, we keep the analysis history in the stack by means of backward-directed pointers



Since the pointers can be modeled as bounded integers, the *PDA* still has a finite stack alphabet

SYSTEMATIC CONSTRUCTION OF THE BOTTOM-UP SYNTAX ANALYZER

shortly called *PDA*

- construction of the pilot graph
  - the pilot drives (pilots) the *PDA*
  - in each macro-state (m-state) the pilot incorporates all the information about any possible phrase form that reaches the m-state
    - each m-state contains machine states with look-ahead (prospection)
      - look-ahead: which chars we expect to see in the input at reduction time; it corresponds to the followers *follow* (*A*) we have seen before
- the m-states are used to build a few analysis threads in the stack, which correspond to possible derivations
  - ≡ computations of the machine network
  - ≡ paths with ε-arcs at each machine change, labeled with the scanned string
- verification of determinism conditions on the pilot graph: three problems
  - *shift-reduce* conflict
  - *reduce-reduce* conflict
  - (less frequently) two or more paths that merge into one state with non-disjoint look-ahead: *convergence conflict*
- if the determinism test is passed, the *PDA* can analyze the string deterministically
- the *PDA* uses the information stored in the pilot graph and in the stack

A FEW DEFINITIONS

SET OF INITIALS – similar (but not identical) to Berry-Sethi

Set of chars found starting from state q_A of machine M_A of the net M

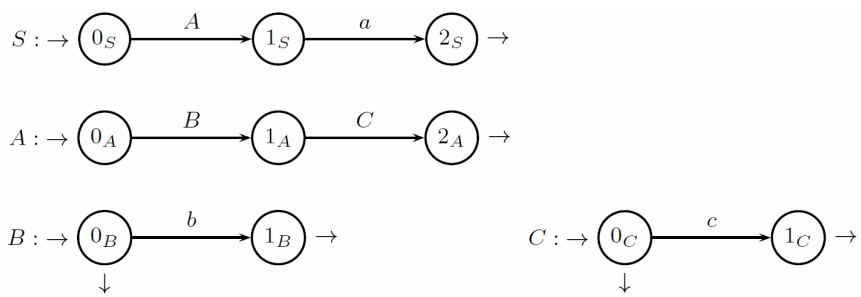
$$Ini(q_A) = Ini\left(L(q_A)\right) = \{a \in \Sigma \mid a\Sigma^* \cap L(q_A) \neq \emptyset\}$$

Defined in the three cases aside *by logical clauses*

1. $a \in Ini(q_A)$ if $\exists$ arc $q_A \xrightarrow{a} r_A$
2. $a \in Ini(q_A)$ if $\exists$ arc $q_A \xrightarrow{B} r_A$
   $\wedge\ a \in Ini(0_B)$ (where 0_B is the initial state of machine M_B)
3. $a \in Ini(q_A)$ if $\exists$ arc $q_A \xrightarrow{B} r_A$
   $\wedge\ L(0_B)$ is nullable $\wedge\ a \in Ini(r_A)$

EXAMPLE

$S \to Aa \qquad A \to BC \qquad B \to b \mid \varepsilon \qquad C \to c \mid \varepsilon$



$$Ini(0_S) = Ini(0_A) \cup \overbrace{Ini(1_S)}^{\{a\}} \text{ because } L(0_A) \text{ is nullable}$$

$$Ini(0_A) = \overbrace{Ini(0_B)}^{\{b\}} \cup Ini(1_A) \text{ because } L(0_B) \text{ is nullable}$$

$$Ini(1_A) = \overbrace{Ini(0_C)}^{\{c\}} \cup \overbrace{Ini(2_A)}^{\emptyset} \text{ because } L(0_C) \text{ is nullable}$$

$$Ini(0_S) = \{b\} \cup \{c\} \cup \{a\}$$

ITEM (or *candidate*) $\langle q_B, a \rangle$ in $Q \times (\Sigma \cup \{ \dashv \})$

Two or more items with the same state can be grouped into one item

$$\{ \langle q, a_1 \rangle, \langle q, a_2 \rangle, \ldots \langle q, a_k \rangle \} \quad \Rightarrow \quad \langle q, \{ a_1, a_2, \ldots a_k \} \rangle$$

An item with a machine final state is said to be a *reduction item*

CLOSURE

Function *closure* (*C*) computes a kind of closure of a set *C* of items with look-ahead

Repeatedly apply this (recursive) clause until a fixed point is reached

$$closure(C) = C$$

initial setting

recursive step

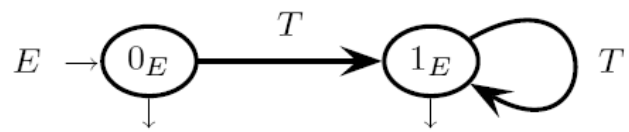
$$\langle 0_B, b \rangle \in closure(C) \quad \text{if} \quad \begin{cases} \exists \text{ candidate } \langle q, a \rangle \in C & \text{and} \\ \exists \text{ arc } q \xrightarrow{B} r \text{ in } \mathcal{M} & \text{and} \\ b \in Ini(\, L(r) \cdot a \,) \end{cases}$$

"*a*" comes in when *L* (*r*) is nullable

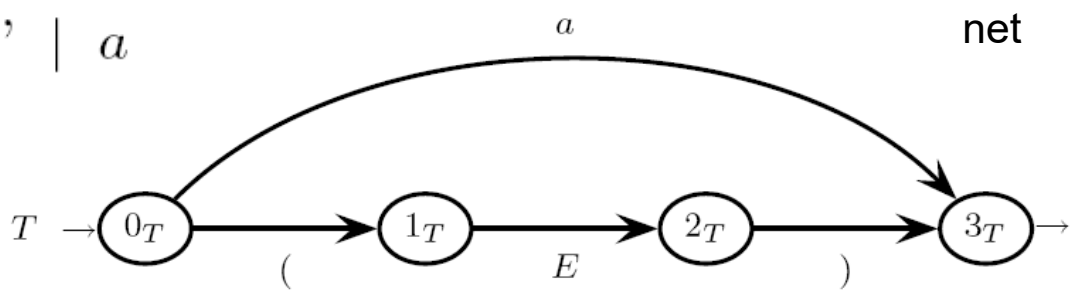
RUNNING EXAMPLE – an *EBNF* grammar

$\Sigma = \{\ a,\ `(',\ `)'\ \}$ $\quad V = \{\ E,\ T\ \}$ axiom *E*

$$P = \begin{cases} E \rightarrow T^* \\ T \rightarrow `(' \ E \ `)' \ \mid \ a \end{cases}$$



net



Sample strings

$\varepsilon \quad a\,a \quad (\ ) \quad (\ a\ ) \quad (\ )\,a \quad (\ a\,a\ ) \quad (\ (\ a\ )\ ) \quad (\ (\ )\,a\,a\ )$

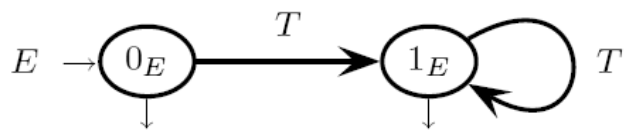
Each machine state p_A of a m-state *I* has a look-ahead (or prospection) set $\pi \subseteq \Sigma \cup \{\ \dashv\ \}$

Item (or candidate) $\langle p_A, \pi \rangle \in I$ contains the chars that may follow a string generated by *A*

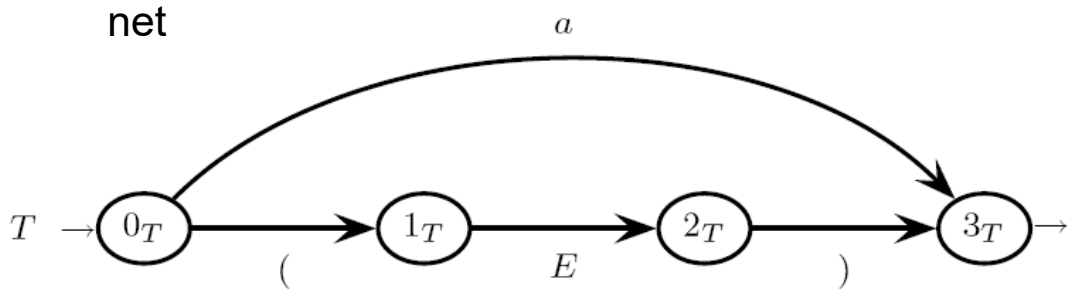
When the analysis reaches the end of a string generated by a machine M_A,
the *PDA* makes a reduction with current char *cc* $\in$ look-ahead, i.e., *cc* $\in \pi$

CLOSURE EXAMPLE – for the running example

$$\Sigma = \{\, a,\ `(',\ `)' \,\} \qquad V = \{\, E,\ T \,\} \qquad P = \begin{cases} E \to T^* \\ T \to `(' \ E \ `)' \ \mid \ a \end{cases}$$



net



C	$function\ closure(C)$			
$\langle 0_E, \dashv \rangle$	$\langle 0_E, \dashv \rangle$	$\langle 0_T, \{\, a, `(', \dashv \,\} \rangle$		
$\langle 1_T, \dashv \rangle$	$\langle 1_T, \dashv \rangle$	$\langle 0_E, `)' \rangle$	$\langle 0_T, \{\, a, `(', `)' \,\} \rangle$	

SHIFT OPERATION – also denoted θ

We are close to defining an algorithm for constructing the pilot graph

Define the *shift* operation θ

with a *shift* operation the item look-ahead does not change – a new look-ahead is created by *closure*

$$\begin{cases} \vartheta\,(\,\langle\, p_A,\, \rho\,\rangle\,,\, X\,) = \langle\, q_A,\, \rho\,\rangle & \text{if the arc } p_A \xrightarrow{X} q_A \text{ exists} \\ \text{the empty set} & \text{otherwise} \end{cases}$$

A *shift* corresponds to a transition in a machine *Y*

- if *X* = *c* is a terminal symbol, then *shift* is a *PDA* move that reads a char *c* in the input
- if *X* is a non-terminal symbol, then *shift* is a *PDA* ε-move after a reduction $z \to X$ and it does not read any input
- machine *Y* runs a transition with non-terminal label *X*
- the analysis goes on after recognizing an input substring $z \in L\,(X)$ derivable from the non-terminal *X*

The shift operation extends to sets of items (i.e., *m-states*) as obvious

$$\vartheta\,(\,C,\, X\,) = \bigcup_{\forall\ \text{candidate}\ \gamma \in C} \vartheta\,(\gamma, X)$$

HOW TO BUILD THE PILOT GRAPH

A *macro-state* (shortly *m-state*) is defined as a set of items

The pilot is a DFA, named $\mathcal{P}$, defined by the following entities:

- the set R of m-states
- the *pilot alphabet* is the union $\Sigma \cup V$ of the terminal and nonterminal alphabets, to be also named the *grammar symbols*
- the initial m-state, I_0, is the set: $I_0 = closure\,(\langle 0_S, \dashv \rangle)$
- the m-state set $R = \{ I_0, I_1, \ldots \}$ and the state-transition function $\vartheta\colon R \times (\Sigma \cup V) \to R$ are computed starting from I_0

OBSERVATIONS AND TERMINOLOGY

Incremental construction: it ends when nothing changes any longer

No final m-state: the pilot does not recognize strings, it controls the *PDA*

The items in each m-state *I* of the pilot are parted into two groups

base, contains the items obtained after a shift $\Rightarrow$ non-initial states

closure, contains the items obtained after a closure $\Rightarrow$ initial states

Call *kernel* of a m-state *I* the set of the m-states of *I* without look-ahead

PILOT GRAPH CONSTRUCTION ALGORITHM

```
R' := { I_0 }                          - - prepare the initial m-state I_0
- - loop that updates the m-state set and the state-transition function
do
    R := R'                            - - update the m-state set R
    - - loop that computes possibly new m-states and arcs
    for (each m-state I ∈ R and symbol X ∈ Σ ∪ V) do
        - - compute the base of a m-state and its closure
        I' := closure(ϑ (I, X))
        - - check if the m-state is not empty and add the arc
        if (I' ≠ ∅) then
            add arc I --X--> I' to the graph of ϑ
            - - check if the m-state is a new one and add it
            if (I' ∉ R) then
                add m-state I' to the set R'
            end if
        end if
    end for
while (R ≠ R')                         - - repeat if the graph has grown
```

RUNNING EXAMPLE – FIRST THREE M-STATES

I_0 = *closure* $(\langle 0_E, \{\dashv\}\rangle) = \{ \langle 0_E, \{\dashv\}\rangle, \langle 0_T, \{a, (, \{\dashv\}\rangle \}$

for the item $\langle 0_E, \{\dashv\}\rangle$ of I_0 and the T-arc $0_E \rightarrow 1_E$ do

compute *shift* $(I_0, T) = \{ \langle 1_E, \{\dashv\}\rangle \}$

compute *closure* $(\langle 1_E, \{\dashv\}\rangle) = \{ \langle 1_E, \{\dashv\}\rangle, \langle 0_T, \{a, (, \dashv\}\rangle \}$

add to the pilot a new m-state I_1 and a new T-arc $I_0 \rightarrow I_1$

for the item $\langle 0_T, \{a, (, \dashv\}\rangle$ of I_0 and the a-arc $0_T \rightarrow 3_T$ do

compute *shift* $(I_0, a) = \{ \langle 3_T, \{a, (, \dashv\}\rangle \}$

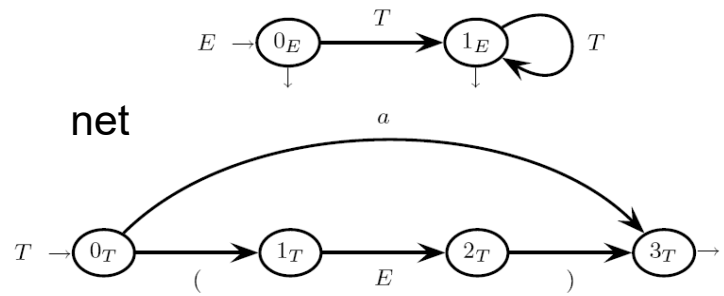
compute *closure* $(\langle 3_T, \{a, (, \dashv\}\rangle) = \{ \langle 3_T, \{a, (, \dashv\}\rangle \}$

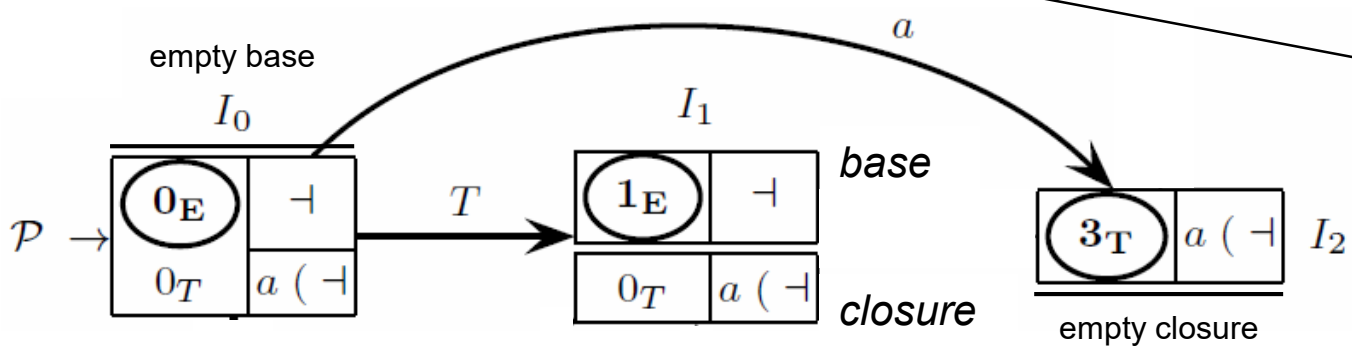
add to the pilot a new m-state I_2 and a new a-arc $I_0 \rightarrow I_2$

$$E \rightarrow T^*$$

$$T \rightarrow \text{`('}\ E\ \text{`)'} \mid a$$

net





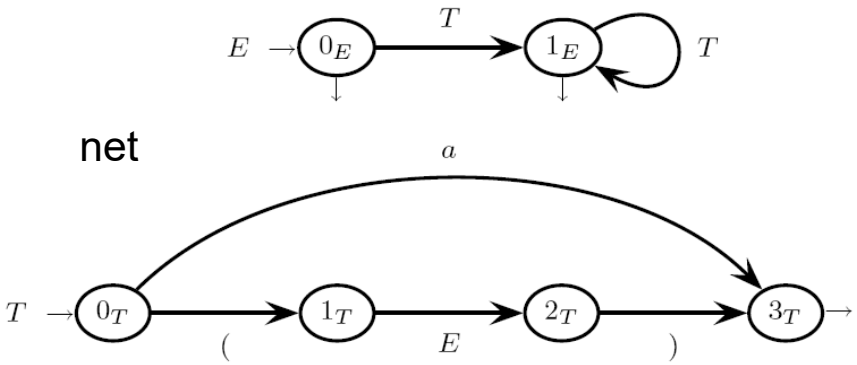
The machine final states in the pilot m-states are highlighted and encircled

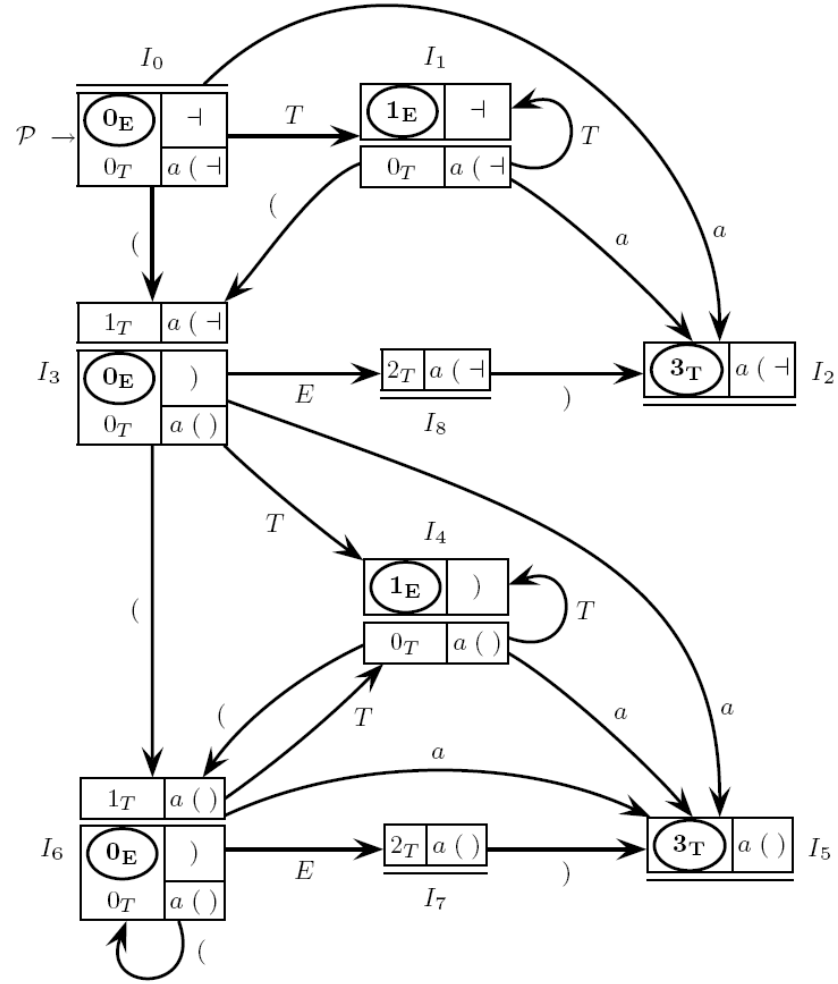
The initial m-state I_0 only consists of a closure – its base is empty by construction

Base and closure are graphically divided by a double line – the closure of I_2 is empty

A new look-ahead is created by a closure – a shift just inherits the previous look-ahead

COMPLETE PILOT GRAPH





pilot graph

State pairs with the same kernel (kernel-equivalent):

(I_3, I_6) (I_1, I_4) (I_2, I_5) (I_7, I_8)

If an m-state contains an item with a final state, then the *PDA* makes a reduction move

The look-ahead of the reduction item indicates the input characters expected at reduction time

The *PDA* verifies the current input char *cc*

- if *cc* ∈ look-ahead, *PDA* makes a reduction
- else (if possible) *PDA* reads input char *cc* and makes a shift on an arc with label *cc*
- otherwise *PDA* stops and rejects